

A 股明星分析师预期上修共识策略系统

低成本个人 MVP 需求与技术实施文档

1. 项目背景

本项目开发一个本地运行的 A 股周度选股系统。

策略逻辑是：

跟踪 A 股优秀卖方分析师近期看好的公司，尤其是新财富、水晶球、金牛等上榜明星分析师。如果一只股票在最近 14 天内被两家以上不同券商的明星分析师同时看好，并且目标价或盈利预测出现上修，同时股价尚未充分上涨，则认为该股票存在“预期上修但市场尚未完全反应”的机会。

第一版目标是个人研究和模拟跟踪，不作为投资建议，不自动交易。

2. 项目目标

开发一个 Python 本地系统，每周运行一次，输出：

1. 最近 14 天明星分析师有效研报池；
 2. 符合条件的候选股票池；
 3. Top20 候选股票；
 4. 8 只建议组合；
 5. 每只股票的得分拆解；
 6. 入选理由；
 7. 剔除原因；
 8. 数据质量报告；
 9. Markdown、CSV、Excel 格式周报。
-

3. 第一版原则

第一版遵循以下原则：

1. 低成本优先；
2. 免费数据源优先；
3. 能跑通优先；
4. 不追求一开始全自动准确；
5. 允许人工维护明星分析师名单；
6. 允许人工复核最终结果；
7. 所有规则可解释；

8. 所有关键数据保留 source 字段；
9. 所有失败接口记录日志；
10. 不自动下单。

4. 策略名称

明星分析师预期上修共识策略。

英文项目名：

analyst_consensus_strategy

5. 运行时间设计

默认运行节奏：

周日 20:00 北京时间：主运行，生成下周候选组合
周一 08:30 北京时间：可选增量确认
周一 09:25 北京时间：人工复核集合竞价、高开、停牌、涨停
周一 09:35 以后：人工决定是否买入

第一版只实现手工命令运行。

后续可加入 APScheduler 定时任务。

6. 投资策略逻辑

核心逻辑：

买入的不是单篇研报，也不是最高目标价，
而是“明星分析师共识上修 + 股价尚未充分反应”的预期差。

一只股票要进入候选池，必须尽量满足：

1. 最近 14 天内有明星分析师看多；
2. 至少 2 家不同券商；
3. 至少 2 位不同明星分析师或明星团队；
4. 评级为买入、增持、强烈推荐、推荐；

5. 有明确目标价；
6. 目标价相比当前价格有至少 25% 空间；
7. 股价在研报发布后没有涨太多；
8. 公司不是 ST，不是刚上市新股；
9. 成交额足够；
10. 没有严重风险公告；
11. 行业不过度集中。

7. 数据源设计

7.1 数据源优先级

第一版数据源：

免费默认：

AKShare

可选增强：

Tushare Pro

人工维护：

明星分析师名单 CSV

股票黑名单 CSV

券商别名 YAML

分析师别名 CSV

系统必须做到：

没有 Tushare Token，也可以运行。

有 Tushare Token，则优先使用 Tushare 补充数据。

7.2 需要的数据类型

系统需要以下数据：

数据类型	用途	第一版来源
股票基础信息	股票代码、名称、上市日期、ST 判断	AKShare / Tushare
日行情	涨跌幅、MA20、MA60、研报后涨幅	AKShare / Tushare

数据类型	用途	第一版来源
成交额/估值/市值	流动性和估值过滤	AKShare / Tushare daily_basic
研报/评级/目标价	核心策略信号	AKShare
盈利预测	EPS/净利润上修	AKShare
明星分析师名单	判断是否权威分析师	手工 CSV
风险事件/公告	剔除重大风险	AKShare / 公开公告标题
指数行情	大盘风控	AKShare / Tushare

8. 推荐项目结构

```

analyst_consensus_strategy/
├── README.md
├── requirements.txt
├── pyproject.toml
├── .env.example
├── config/
│   ├── strategy.yaml
│   ├── broker_alias.yaml
│   ├── rating_map.yaml
│   └── risk_keywords.yaml
├── data/
│   ├── strategy.db
│   ├── raw/
│   ├── manual/
│   │   ├── analyst_awards_seed.csv
│   │   ├── analyst_alias.csv
│   │   └── stock_blacklist.csv
│   └── output/
├── logs/
├── outputs/
│   └── weekly/
├── src/
│   ├── __init__.py
│   ├── cli.py
│   ├── settings.py
│   ├── db.py
│   ├── models.py
│   ├── utils/
│   │   ├── __init__.py
│   │   └── dates.py

```

```
| | | | | codes.py
| | | | | normalize.py
| | | | | └─ logger.py
| | | └─ data_providers/
| | | | | └─ __init__.py
| | | | | └─ akshare_provider.py
| | | | | └─ tushare_provider.py
| | | | | └─ manual_provider.py
| | └─ pipeline/
| | | | | └─ __init__.py
| | | | | └─ update_stock_universe.py
| | | | | └─ update_market_data.py
| | | | | └─ update_reports.py
| | | | | └─ update_forecasts.py
| | | | | └─ update_risk_events.py
| | | | | └─ weekly_run.py
| | └─ strategy/
| | | | | └─ __init__.py
| | | | | └─ filters.py
| | | | | └─ factors.py
| | | | | └─ scorer.py
| | | | | └─ portfolio.py
| | | | | └─ report_generator.py
| | └─ dashboard/
| | | | | └─ __init__.py
| | | | | └─ app.py
└─ tests/
| | └─ test_normalize.py
| | └─ test_filters.py
| | └─ test_factors.py
| | └─ test_scorer.py
| | └─ test_portfolio.py
```

9. Python 依赖

requirements.txt :

```
pandas
numpy
akshare
tushare
requests
beautifulsoup4
```

```
lxml
sqlalchemy
python-dotenv
pydantic
pyyaml
typer
rich
openpyxl
xlsxwriter
apscheduler
streamlit
pytest
```

Streamlit 是可选依赖。第一版即使 Streamlit 不完成，核心 CLI 也必须可运行。

10. 环境变量

`.env.example` :

```
TUSHARE_TOKEN=
USE_TUSHARE=false
USE_AKSHARE=true
DB_URL=sqlite:///data/strategy.db
TIMEZONE=Asia/Shanghai
DEFAULT_MARKET_INDEX=000300.SH
```

要求：

1. 不允许把 Tushare Token 写死在代码中；
2. 如果 `TUSHARE_TOKEN` 为空，自动关闭 Tushare；
3. 如果 Tushare 接口失败，自动降级到 AKShare；
4. 运行日志要提示当前使用的数据源。

11. 配置文件

11.1 `config/strategy.yaml`

```
lookback_report_days: 14
market_data_days: 180
risk_event_days: 90
```

```
min_target_upside: 0.25
max_target_upside: 1.00

min_avg_amount_20d: 150000000
max_post_report_return: 0.12
min_post_report_return: -0.08

max_return_20d: 0.35
max_return_60d: 0.80

min_listing_days: 365

portfolio_size: 8
max_per_industry: 2
min_industry_count: 4

market_risk:
  index_code: "000300"
  ma60_position: 0.50
  ma120_position: 0.30
  normal_position: 1.00

weights:
  target_upside: 0.25
  star_consensus: 0.20
  target_revision: 0.20
  earnings_revision: 0.15
  price_reaction: 0.10
  trend: 0.10
```

11.2 config/rating_map.yaml

```
强烈推荐: buy
买入: buy
推荐: buy
增持: overweight
优于大市: overweight
谨慎推荐: overweight
审慎增持: overweight
中性: neutral
持有: neutral
观望: neutral
减持: underweight
```

```
回避：underweight  
卖出：sell
```

只有以下评级允许进入候选池：

```
buy  
overweight
```

11.3 config/broker_alias.yaml

```
中信建投：中信建投证券  
中信建投证券：中信建投证券  
广发：广发证券  
广发证券：广发证券  
长江：长江证券  
长江证券：长江证券  
国盛：国盛证券  
国盛证券：国盛证券  
申万宏源：申万宏源  
申万宏源证券：申万宏源  
东吴：东吴证券  
东吴证券：东吴证券  
华创：华创证券  
华创证券：华创证券  
浙商：浙商证券  
浙商证券：浙商证券  
平安：平安证券  
平安证券：平安证券  
天风：天风证券  
天风证券：天风证券  
中泰：中泰证券  
中泰证券：中泰证券  
国联：国联证券  
国联证券：国联证券
```

11.4 config/risk_keywords.yaml

```
severe:  
- 立案调查  
- 重大违法
```


- 退市风险
- 暂停上市
- 终止上市
- 非标审计
- 无法表示意见
- 保留意见
- 资金占用
- 违规担保
- 债务逾期

medium:

- 问询函
- 监管函
- 减持
- 大股东减持
- 董监高减持
- 业绩预告下修
- 业绩修正
- 亏损
- 商誉减值
- 重大诉讼
- 股权质押

12. 手工维护数据

12.1 明星分析师名单

路径：

```
data/manual/analyst_awards_seed.csv
```

字段：

```
analyst_name,broker,team_name,industry,award_name,award_year,rank,source_note,active
```

初始示例：

```
analyst_name,broker,team_name,industry,award_name,award_year,rank,source_note,active
邬博华,长江证券,邬博华团队,电力设备,水晶球,2024,1,初始种子名单,1
曹海花,长江证券,邬博华团队,电力设备,水晶球,2024,1,初始种子名单,1
曾朵红,东吴证券,曾朵红团队,电力设备,水晶球,2024,2,初始种子名单,1
```

王芳,中泰证券,王芳团队,电子,水晶球,2024,1,初始种子名单,1
耿正,广发证券,耿正团队,电子,水晶球,2024,2,初始种子名单,1
乐加栋,广发证券,乐加栋团队,房地产,水晶球,2024,1,初始种子名单,1
马莉,浙商证券,马莉团队,纺织服饰/轻工造纸,水晶球,2024,1,初始种子名单,1
范超,长江证券,范超团队,非金属建材,水晶球,2024,1,初始种子名单,1
陈福,广发证券,陈福团队,非银行金融,水晶球,2024,1,初始种子名单,1
王鹤涛,长江证券,王鹤涛团队,钢铁/有色金属,水晶球,2024,1,初始种子名单,1
张韦华,长江证券,张韦华团队,公用事业,水晶球,2024,1,初始种子名单,1
郭鹏,广发证券,郭鹏团队,环保/公用事业,水晶球,2024,1,初始种子名单,1
赵智勇,长江证券,赵智勇团队,机械,水晶球,2024,1,初始种子名单,1
代川,广发证券,代川团队,机械,水晶球,2024,2,初始种子名单,1
马昕晔,申万宏源,马昕晔团队,基础化工/石油化工,水晶球,2024,1,初始种子名单,1
刘高畅,国盛证券,刘高畅团队,计算机,水晶球,2024,1,初始种子名单,1
宗建树,长江证券,宗建树团队,计算机,水晶球,2024,2,初始种子名单,1
管泉森,国联证券,管泉森团队,家电,水晶球,2024,1,初始种子名单,1
何亚轩,国盛证券,何亚轩团队,建筑工程,水晶球,2024,1,初始种子名单,1
韩轶超,长江证券,韩轶超团队,交通运输,水晶球,2024,1,初始种子名单,1
孟祥杰,广发证券,孟祥杰团队,军工,水晶球,2024,1,初始种子名单,1
张津铭,国盛证券,张津铭团队,煤炭,水晶球,2024,1,初始种子名单,1
陈佳,长江证券,陈佳团队,农林牧渔,水晶球,2024,1,初始种子名单,1
李锦,长江证券,李锦团队,批发零售,水晶球,2024,1,初始种子名单,1
高伊楠,长江证券,高伊楠团队,汽车及零部件,水晶球,2024,1,初始种子名单,1
赵刚,长江证券,赵刚团队,社会服务,水晶球,2024,1,初始种子名单,1
魏凯,长江证券,魏凯团队,石油化工,水晶球,2024,1,初始种子名单,1
欧阳予,华创证券,欧阳予团队,食品饮料,水晶球,2024,1,初始种子名单,1
董广阳,华创证券,欧阳予团队,食品饮料,水晶球,2024,1,初始种子名单,1
宋嘉吉,国盛证券,宋嘉吉团队,通信,水晶球,2024,1,初始种子名单,1
唐海清,天风证券,唐海清团队,通信,水晶球,2024,2,初始种子名单,1
叶寅,平安证券,叶寅团队,医药生物,水晶球,2024,1,初始种子名单,1
倪亦道,平安证券,叶寅团队,医药生物,水晶球,2024,1,初始种子名单,1
孙媛媛,兴业证券,孙媛媛团队,医药生物,水晶球,2024,4,初始种子名单,1
梁凤洁,浙商证券,梁凤洁团队,银行,水晶球,2024,1,初始种子名单,1
邱冠华,浙商证券,梁凤洁团队,银行,水晶球,2024,1,初始种子名单,1
吴立,天风证券,吴立团队,中小盘/新兴产业,水晶球,2024,1,初始种子名单,1
孔蓉,天风证券,吴立团队,中小盘/新兴产业,水晶球,2024,1,初始种子名单,1

要求：

1. 系统必须支持后续追加更多分析师；
2. 同一分析师可以有多个年份、多个奖项、多个行业；
3. 分析师匹配优先使用姓名 + 券商；
4. 如果只匹配姓名，不匹配券商，则标记为 uncertain_match；
5. 不确定匹配不进入正式明星分析师计数，只进入人工复核字段。

12.2 分析师别名表

路径：

```
data/manual/analyst_alias.csv
```

字段：

```
raw_name,standard_name,broker,notes
```

示例：

```
raw_name,standard_name,broker,notes
邬博华团队,邬博华,长江证券,团队名映射
欧阳予团队,欧阳予,华创证券,团队名映射
叶寅团队,叶寅,平安证券,团队名映射
```

12.3 股票黑名单

路径：

```
data/manual/stock_blacklist.csv
```

字段：

```
stock_code,stock_name,reason,start_date,end_date,active
```

示例：

```
stock_code,stock_name,reason,start_date,end_date,active
000000,示例股票,人工排除示例,20260101,,0
```

13. 数据库设计

数据库使用 SQLite。

默认路径：

data/strategy.db

13.1 stock_master

```
CREATE TABLE IF NOT EXISTS stock_master (  
    stock_code TEXT PRIMARY KEY,  
    ts_code TEXT,  
    stock_name TEXT,  
    exchange TEXT,  
    list_date TEXT,  
    delist_date TEXT,  
    industry TEXT,  
    list_status TEXT,  
    is_st INTEGER DEFAULT 0,  
    updated_at TEXT  
);
```

13.2 daily_bar

```
CREATE TABLE IF NOT EXISTS daily_bar (  
    stock_code TEXT,  
    trade_date TEXT,  
    open REAL,  
    high REAL,  
    low REAL,  
    close REAL,  
    pre_close REAL,  
    volume REAL,  
    amount REAL,  
    adj_factor REAL,  
    source TEXT,  
    updated_at TEXT,  
    PRIMARY KEY (stock_code, trade_date)  
);
```

13.3 daily_basic

```
CREATE TABLE IF NOT EXISTS daily_basic (  
    stock_code TEXT,  
    trade_date TEXT,  
    close REAL,  
    turnover_rate REAL,  
    volume_ratio REAL,  
    pe REAL,  
    pe_ttm REAL,  
    pb REAL,  
    ps REAL,  
    total_mv REAL,  
    circ_mv REAL,  
    source TEXT,  
    updated_at TEXT,  
    PRIMARY KEY (stock_code, trade_date)  
);
```

13.4 research_rating

```
CREATE TABLE IF NOT EXISTS research_rating (  
    id TEXT PRIMARY KEY,  
    stock_code TEXT,  
    stock_name TEXT,  
    publish_date TEXT,  
    broker TEXT,  
    analyst_raw TEXT,  
    rating TEXT,  
    normalized_rating TEXT,  
    rating_change TEXT,  
    previous_rating TEXT,  
    target_price_low REAL,  
    target_price_high REAL,  
    target_price REAL,  
    source TEXT,  
    source_url TEXT,  
    created_at TEXT,  
    updated_at TEXT  
);
```

13.5 research_report

```
CREATE TABLE IF NOT EXISTS research_report (  
  id TEXT PRIMARY KEY,  
  stock_code TEXT,  
  stock_name TEXT,  
  report_title TEXT,  
  broker TEXT,  
  rating TEXT,  
  normalized_rating TEXT,  
  report_count_1m INTEGER,  
  eps_y0 REAL,  
  eps_y1 REAL,  
  eps_y2 REAL,  
  pe_y0 REAL,  
  pe_y1 REAL,  
  pe_y2 REAL,  
  industry TEXT,  
  publish_date TEXT,  
  pdf_url TEXT,  
  source TEXT,  
  created_at TEXT,  
  updated_at TEXT  
);
```

13.6 profit_forecast_snapshot

```
CREATE TABLE IF NOT EXISTS profit_forecast_snapshot (  
  id TEXT PRIMARY KEY,  
  snapshot_date TEXT,  
  stock_code TEXT,  
  stock_name TEXT,  
  source TEXT,  
  report_count INTEGER,  
  buy_count REAL,  
  overweight_count REAL,  
  neutral_count REAL,  
  underweight_count REAL,  
  sell_count REAL,  
  eps_year_0 REAL,  
  eps_year_1 REAL,  
  eps_year_2 REAL,  
  np_year_0 REAL,
```

```
np_year_1 REAL,  
np_year_2 REAL,  
institution_count INTEGER,  
created_at TEXT  
);
```

13.7 analyst_awards

```
CREATE TABLE IF NOT EXISTS analyst_awards (  
  id TEXT PRIMARY KEY,  
  analyst_name TEXT,  
  broker TEXT,  
  team_name TEXT,  
  industry TEXT,  
  award_name TEXT,  
  award_year INTEGER,  
  rank TEXT,  
  source_note TEXT,  
  active INTEGER DEFAULT 1,  
  created_at TEXT,  
  updated_at TEXT  
);
```

13.8 risk_event

```
CREATE TABLE IF NOT EXISTS risk_event (  
  id TEXT PRIMARY KEY,  
  stock_code TEXT,  
  stock_name TEXT,  
  event_date TEXT,  
  event_type TEXT,  
  title TEXT,  
  url TEXT,  
  severity TEXT,  
  keyword_hit TEXT,  
  source TEXT,  
  created_at TEXT  
);
```

13.9 weekly_candidate

```
CREATE TABLE IF NOT EXISTS weekly_candidate (  
  id TEXT PRIMARY KEY,  
  run_date TEXT,  
  stock_code TEXT,  
  stock_name TEXT,  
  industry TEXT,  
  latest_close REAL,  
  target_price REAL,  
  target_upside REAL,  
  effective_star_analyst_count INTEGER,  
  effective_broker_count INTEGER,  
  target_price_revision REAL,  
  earnings_revision REAL,  
  post_report_return REAL,  
  trend_score REAL,  
  risk_penalty REAL,  
  upside_score REAL,  
  consensus_score REAL,  
  target_revision_score REAL,  
  earnings_revision_score REAL,  
  price_reaction_score REAL,  
  total_score REAL,  
  rank INTEGER,  
  pass_filter INTEGER,  
  fail_reason TEXT,  
  created_at TEXT  
);
```

13.10 portfolio_suggestion

```
CREATE TABLE IF NOT EXISTS portfolio_suggestion (  
  id TEXT PRIMARY KEY,  
  run_date TEXT,  
  stock_code TEXT,  
  stock_name TEXT,  
  industry TEXT,  
  weight REAL,  
  rank INTEGER,  
  total_score REAL,  
  target_upside REAL,  
  reason TEXT,
```



```
        created_at TEXT
    );
```

13.11 data_quality_report

```
CREATE TABLE IF NOT EXISTS data_quality_report (
    id TEXT PRIMARY KEY,
    run_date TEXT,
    item TEXT,
    value TEXT,
    severity TEXT,
    message TEXT,
    created_at TEXT
);
```

14. 核心筛选规则

14.1 基础股票池

保留：

```
A 股
非 ST / 非 *ST
上市满 1 年
最近 20 个交易日日均成交额 >= 1.5 亿
最近一期净利润或扣非净利润 > 0
```

剔除：

```
ST / *ST / 退市整理
上市未满 1 年
近 20 日均成交额 < 1.5 亿
最近一期净利润 < 0
人工黑名单 active = 1
```

如果财务数据缺失：

第一版不直接剔除，但 fail_warning 中提示“财务数据缺失，需要人工复核”。

14.2 研报条件

最近 14 天内满足：

至少 2 家不同券商
至少 2 位不同明星分析师或明星团队
评级为 buy / overweight
有明确目标价
目标价空间 $\geq 25\%$
目标价空间 $\leq 100\%$

如果只找到 1 位明星分析师，但有 2 家券商普通分析师看多：

进入观察池，不进入正式组合。

14.3 核心过滤

剔除：

目标价空间 $< 25\%$
目标价空间 $> 100\%$
研报发布后至选股日涨幅 $> 12\%$
近 20 个交易日涨幅 $> 35\%$
近 60 个交易日涨幅 $> 80\%$
最近 60 日出现严重风险事件

降权：

研报发布后至选股日涨幅 $< -8\%$
个股收盘价低于 MA60
最近 30 日出现中等风险事件
财务字段缺失
分析师匹配不确定

15. 因子计算

15.1 目标价空间

```
target_upside = target_price / latest_close - 1
```

多篇有效研报时：

使用最近 14 天有效明星分析师目标价中位数。
不要使用最高目标价。

15.2 有效明星共识数量

统计规则：

同一券商同一团队只算 1 个
同一分析师多篇报告只算 1 个
不同券商才算独立共识

字段：

```
effective_star_analyst_count  
effective_broker_count
```

正式门槛：

```
effective_broker_count >= 2
```

15.3 目标价上修

优先级：

1. 同一股票 + 同一券商 + 同一分析师：本次目标价 vs 上一次目标价
2. 同一股票 + 同一券商：本次目标价 vs 上一次目标价

3. 同一股票过去 90 天明星分析师目标价中位数
4. 若无历史, 则设为中性分 0.5

公式：

$$\text{target_price_revision} = \text{current_target_price} / \text{previous_target_price} - 1$$

15.4 盈利预测上修

第一版使用每周快照比较。

每周保存 profit_forecast_snapshot。
从第二周开始, 比较本周 EPS / 净利润预测均值 与 上周或四周前快照。

优先使用：

$$\text{earnings_revision} = \text{current_eps_y1} / \text{previous_eps_y1} - 1$$

如果 EPS 不可用, 使用：

$$\text{earnings_revision} = \text{current_np_y1} / \text{previous_np_y1} - 1$$

如果无历史数据：

$$\text{earnings_revision_score} = 0.5$$

15.5 研报后价格反应

定义：

$$\text{post_report_return} = \text{latest_close} / \text{close_before_first_effective_report} - 1$$

过滤：

```
post_report_return > 12%, 剔除  
post_report_return < -8%, 降权
```

价格反应打分：

```
def calc_price_reaction_score(x: float | None) -> float:  
    if x is None:  
        return 0.5  
    if x > 0.12:  
        return 0.0  
    if 0 <= x <= 0.08:  
        return 1.0  
    if 0.08 < x <= 0.12:  
        return 0.6  
    if -0.08 <= x < 0:  
        return 0.7  
    return 0.3
```

15.6 趋势确认

计算：

```
ma20 = close.rolling(20).mean()  
ma60 = close.rolling(60).mean()  
ret20 = close / close.shift(20) - 1  
ret60 = close / close.shift(60) - 1
```

趋势分：

```
close > MA20 : +0.3  
close > MA60 : +0.4  
20日收益率 > 0 : +0.2  
60日收益率 > 0 : +0.1
```

最终限制在 0 到 1。

16. 因子标准化

使用候选池分位数排名，不使用简单 min-max。

```
score = percentile_rank(value)
```

如果字段缺失：

中性分 = 0.5

如果字段异常：

记录 fail_warning, 不直接崩溃。

17. 综合打分

综合得分 =
目标价空间分位 × 0.25
+ 有效明星共识分位 × 0.20
+ 目标价上修分位 × 0.20
+ 盈利预测上修分位 × 0.15
+ 价格未过度反应分 × 0.10
+ 趋势确认分 × 0.10
- 风险惩罚

风险惩罚：

严重风险：直接剔除
中等风险：risk_penalty = 0.10
财务缺失：risk_penalty = 0.05
分析师匹配不确定：risk_penalty = 0.05

18. 组合构建

默认输出 8 只股票。

规则：

组合数量：8
单票权重：12.5%

单行业最多 2 只
最少行业数 4 个

构建步骤：

1. 按 total_score 从高到低排序；
2. 从第一名开始选择；
3. 如果该行业已满 2 只，跳过；
4. 直到选满 8 只；
5. 如果无法满足至少 4 个行业，输出警告；
6. 如果候选股票不足 8 只，不强行补齐，输出实际数量。

19. 大盘风控

使用沪深300指数。

若沪深300 close < MA60, 建议仓位 = 50%
若沪深300 close < MA120, 建议仓位 = 30%
否则建议仓位 = 100%

第一版只输出建议仓位，不自动执行。

20. 每周运行流程

命令：

```
python -m src.cli weekly-run
```

可选参数：

```
python -m src.cli weekly-run --run-date 20260621  
python -m src.cli weekly-run --days 14  
python -m src.cli weekly-run --output-format all
```

流程：

1. 加载配置；
2. 初始化日志；
3. 检查数据库；
4. 导入人工明星分析师名单；
5. 更新股票基础信息；
6. 更新最近 180 天行情；
7. 更新 daily_basic；
8. 更新最近 30 天研报评级；
9. 更新盈利预测快照；
10. 更新最近 90 天风险事件；
11. 生成最近 14 天有效研报池；
12. 匹配明星分析师；
13. 合并行情、财务、研报、盈利预测；
14. 执行过滤规则；
15. 计算因子；
16. 计算综合得分；
17. 生成 Top20；
18. 生成 8 只组合；
19. 生成 Markdown 报告；
20. 生成 CSV；
21. 生成 Excel；
22. 生成数据质量报告。

21. CLI 命令设计

使用 Typer。

必须支持：

```
python -m src.cli init-db
python -m src.cli import-analysts
python -m src.cli update-stock-universe
python -m src.cli update-market --days 180
python -m src.cli update-reports --days 30
python -m src.cli update-forecasts
python -m src.cli update-risk-events --days 90
python -m src.cli weekly-run
python -m src.cli dashboard
```

如果某些接口暂时不可用，命令也不能直接崩溃，应记录日志并输出数据质量警告。

22. 输出文件

每次 weekly-run 生成：

```
outputs/weekly/YYYYMMDD_candidates.csv
outputs/weekly/YYYYMMDD_portfolio.csv
outputs/weekly/YYYYMMDD_report.md
outputs/weekly/YYYYMMDD_report.xlsx
outputs/weekly/YYYYMMDD_data_quality.csv
```

23. Markdown 报告格式

报告文件：

```
outputs/weekly/YYYYMMDD_report.md
```

内容结构：

```
# 明星分析师预期上修共识策略周报
```

```
运行日期：
```

```
数据截止：
```

```
数据源：
```

```
大盘状态：
```

```
建议仓位：
```

```
## 一、本周建议组合
```

```
表格字段：
```

```
排名
```

```
股票代码
```

```
股票名称
```

```
行业
```

```
建议权重
```

```
综合得分
```

```
目标价空间
```

```
明星分析师数量
```

```
券商数量
```

```
目标价上修
```

```
盈利预测上修
```

```
研报后涨幅
```

```
趋势分
```

入选理由

二、Top20 候选池

表格字段：

排名

股票代码

股票名称

行业

综合得分

目标价空间

明星分析师数量

券商数量

目标价上修

盈利预测上修

研报后涨幅

趋势分

是否入选

未入选原因

三、剔除股票列表

表格字段：

股票代码

股票名称

剔除原因

相关字段值

四、数据质量报告

字段：

股票总数

成功获取行情数量

成功获取研报数量

最近 14 天研报数量

有目标价研报数量

匹配明星分析师研报数量

最终候选数量

缺失字段统计

五、风险提示

包括：

严重风险事件

中等风险事件

财务数据缺失

高开追高提示

涨停买不进提示

六、下周操作建议

BUY / WATCH / SKIP

24. 数据质量要求

每次运行必须输出数据质量。

需要统计：

股票总数
成功获取行情股票数
成功获取 daily_basic 股票数
成功获取研报股票数
最近 14 天研报条数
有目标价研报条数
可用评级条数
匹配到明星分析师条数
候选股票数
最终组合股票数
字段缺失统计
数据源失败统计

如果关键数据不足，输出警告：

最近 14 天有目标价的明星分析师研报不足 5 条，本周不建议自动生成组合，仅输出观察池。

25. 错误处理要求

必须实现：

1. 单只股票失败，不中断全局任务；
2. 单个数据源失败，记录日志并继续；
3. 网络失败重试 3 次；
4. 字段缺失时记录 warning；
5. 数据为空时写入 data_quality_report；

- 6. 不允许静默失败；
- 7. 所有异常要有清楚错误信息。

日志路径：

```
logs/YYYYMMDD_run.log
```

26. 关键函数

26.1 股票代码标准化

```
def normalize_stock_code(raw: str) -> str:
    """
    Convert raw stock code to 6-digit A-share code.

    Examples:
    000001.SZ -> 000001
    SZ000001 -> 000001
    600519.SH -> 600519
    """
```

26.2 股票代码转 Tushare

```
def to_tushare_code(code: str) -> str:
    """
    000001 -> 000001.SZ
    600519 -> 600519.SH
    688001 -> 688001.SH
    """
```

26.3 评级标准化

```
def normalize_rating(raw_rating: str, rating_map: dict) -> str:
    """
    Convert Chinese rating to:
```

```
buy / overweight / neutral / underweight / sell / unknown
"""
```

26.4 券商标准化

```
def normalize_broker(raw_broker: str, alias_map: dict) -> str:
    """
    Normalize broker names using broker_alias.yaml.
    """
```

26.5 分析师拆分

```
def split_analyst_names(raw: str) -> list[str]:
    """
    Split analyst field by:
    comma, Chinese comma, slash, semicolon, whitespace, team suffix.
    """
```

26.6 明星分析师匹配

```
def match_star_analyst(
    analyst_names: list[str],
    broker: str,
    industry: str,
    awards_df: pd.DataFrame
) -> list[dict]:
    """
    Matching priority:
    1. exact analyst_name + broker
    2. exact analyst_name + similar broker alias
    3. team_name contains analyst_name
    4. exact analyst_name only -> uncertain_match
    """
```

26.7 目标价空间

```
def calc_target_upside(target_price: float | None, close: float | None) -> float | None:
    if target_price is None or close is None or close <= 0:
        return None
    return target_price / close - 1
```

26.8 价格反应分

```
def calc_price_reaction_score(post_report_return: float | None) -> float:
    if post_report_return is None:
        return 0.5
    if post_report_return > 0.12:
        return 0.0
    if 0 <= post_report_return <= 0.08:
        return 1.0
    if 0.08 < post_report_return <= 0.12:
        return 0.6
    if -0.08 <= post_report_return < 0:
        return 0.7
    return 0.3
```

26.9 趋势分

```
def calc_trend_score(price_df: pd.DataFrame) -> float:
    """
    Use latest close, MA20, MA60, 20d return, 60d return.
    Return 0~1.
    """
```

26.10 分位数打分

```
def percentile_score(series: pd.Series, value: float | None) -> float:
    """
    Return percentile rank score 0~1.
```

```
If value is missing, return 0.5.  
"""
```

27. 单元测试要求

必须写测试：

```
test_normalize_stock_code  
test_to_tushare_code  
test_normalize_rating  
test_normalize_broker  
test_split_analyst_names  
test_match_star_analyst_exact  
test_match_star_analyst_uncertain  
test_calc_target_upside  
test_calc_price_reaction_score  
test_calc_trend_score  
test_percentile_score  
test_filter_st_stock  
test_filter_new_stock  
test_filter_low_liquidity  
test_filter_high_post_report_return  
test_industry_limit  
test_portfolio_size
```

测试命令：

```
pytest
```

28. README 要求

README 必须包含：

1. 项目介绍
2. 策略逻辑
3. 安装方法
4. 配置 .env
5. 如何初始化数据库
6. 如何导入明星分析师名单
7. 如何更新数据

8. 如何运行 weekly-run
9. 输出文件说明
10. 常见错误
11. 数据源限制
12. 风险声明

29. 风险声明

README 和报告中必须包含：

本系统仅用于个人研究和模拟跟踪，不构成投资建议。
系统输出不代表确定收益。
任何买卖决策需要用户自行判断并承担风险。
免费数据源可能存在延迟、缺失、字段变化、错误等问题。

30. 验收标准

第一版完成后必须满足：

1. 可以在 Windows 本地安装依赖；
2. 可以运行 init-db；
3. 可以导入 analyst_awards_seed.csv；
4. 可以更新股票基础数据；
5. 可以更新最近 180 天行情；
6. 可以更新最近 30 天研报/评级数据；
7. 可以保存盈利预测快照；
8. 可以生成 weekly_candidate 表；
9. 可以生成 Top20 候选池；
10. 可以生成 8 只组合或在不足 8 只时给出原因；
11. 每只股票有入选理由；
12. 每只剔除股票有 fail_reason；
13. 可以输出 Markdown、CSV、Excel 报告；
14. 可以输出数据质量报告；
15. 核心单元测试通过；
16. 数据源失败不会导致整个程序崩溃。

31. 开发阶段拆分

Phase 0：项目骨架

任务：

1. 创建目录结构；
2. 创建 requirements.txt；
3. 创建 .env.example；
4. 创建 config 文件；
5. 创建 SQLite 初始化逻辑；
6. 创建 Typer CLI；
7. 创建日志系统。

验收：

```
python -m src.cli init-db
```

Phase 1：数据层

任务：

1. 实现股票基础信息更新；
2. 实现行情更新；
3. 实现 daily_basic 更新；
4. 实现研报评级更新；
5. 实现盈利预测快照；
6. 实现人工明星分析师名单导入；
7. 实现数据源失败降级。

验收：

```
python -m src.cli import-analysts
python -m src.cli update-stock-universe
python -m src.cli update-market --days 180
python -m src.cli update-reports --days 30
python -m src.cli update-forecasts
```

Phase 2：策略层

任务：

1. 实现基础过滤；
2. 实现明星分析师匹配；
3. 实现目标价空间；
4. 实现目标价上修；
5. 实现盈利预测上修；
6. 实现价格反应；
7. 实现趋势分；
8. 实现风险惩罚；
9. 实现综合打分。

验收：

```
pytest
```

Phase 3：每周输出

任务：

1. 实现 weekly-run；
2. 生成 Top20 候选池；
3. 生成 8 只组合；
4. 生成 Markdown 报告；
5. 生成 CSV；
6. 生成 Excel；
7. 生成数据质量报告。

验收：

```
python -m src.cli weekly-run
```

Phase 4：README 和测试

任务：

1. 补充 README ;
2. 补充测试 ;
3. 补充示例输出 ;
4. 检查 Windows 兼容性。

Phase 5 : 可选 Streamlit 面板

任务 :

1. 展示本周组合 ;
2. 展示 Top20 ;
3. 展示剔除原因 ;
4. 展示数据质量 ;
5. 展示得分拆解。

命令 :

```
python -m src.cli dashboard
```

32. 给开发者的最后要求

请优先实现最小可运行闭环 :

```
数据更新 -> 策略计算 -> 候选池 -> 组合 -> 报告
```

不要一开始追求完美数据、完整回测、复杂界面。

第一版只要每周能稳定输出 :

```
Top20 候选股票  
8 只建议组合  
每只股票的入选理由  
每只股票的剔除原因  
数据质量报告
```

就算 MVP 成功。